University of Colombo School of Computing

SCS 1308 – Foundations of Algorithms

Tutorial - 02

---

1. Consider an algorithm that recursively calculates the sum of all elements in an array of **n** integers. The algorithm adds the first element to the sum of the remaining **n−1** elements. If there are no elements left to process ($n = 0$), the algorithm returns 0.

    a. Identify the **base case** and **recursive case** of the algorithm.

    b. Write the **pseudocode** for the recursive algorithm.

    c. Define a **recurrence relation** for the running time of the algorithm in terms of n.

2. Consider the following recursive algorithm for processing a sequence of n elements.

```
Process(n)
   if n <= 1
         perform 5 constant-time operations
      return
   perform 5 constant-time operations
   Process(n - 1)
```

    a. Derive the recurrence equation $T(n)$, including the base case.

    c. Solve the recurrence using the recursion tree.

    d. Determine the asymptotic time complexity.

3. Consider the following recursive algorithm(assuming n is a positive integer):

```
Process(n)
   if n <= 1
         perform 1 constant-time operation
      return
   perform 5 constant-time operations
   Process(n -1)
   Process(n -1)
```

Answer the following:

a. Derive the recurrence equation $T(n)$, including the base case.

b. Solve the recurrence using the recursion tree.

c. Determine the asymptotic time complexity.

4. A large dataset contains n records that need to be analyzed. To make the analysis more efficient, a data analysis program divides the dataset into **three equal-sized subsets**. The same analysis procedure is then applied recursively to each subset.

   Once the three subsets have been processed, the program **combines the three results** to produce the final analysis result. Assume that combining the results and divides the data set requires time proportional to the number of records being processed, i.e., **c_3n time.**

   The recursion stops when a subset contains only **one record**.

   Assume that n is a power of three.

   a. Design a recursive algorithm based on this description.

   b. Derive the recurrence equation for its running time.

   c. Draw the recursion tree and label the work performed at each level.

   d. Determine the height of the recursion tree.

   e. Find the total work performed at each level and derive the overall time complexity.

   Assume that n is a power of three and that the combining operation takes $\Theta(n)$ time.

5. Two algorithms solve the same problem.
   - Algorithm A: Solves one subproblem of size $n - 1$ and performs constant additional work.
   - Algorithm B: Solves two subproblems of size $n/2$ and performs constant additional work.

   a. Write the recurrence equation for each algorithm.

   b. Solve both recurrences using an appropriate method.

c. Compare their asymptotic time complexities.

d. For a large input, which algorithm has the better theoretical running time? Explain your answer.

e. Discuss why the number and size of recursive calls influence an algorithm's efficiency.

6. A company needs to locate a particular record in a dataset that is too large to examine efficiently using a sequential approach. The records can be divided into smaller groups, and additional information is available to determine which group may contain the required record.

   Design a recursive strategy to locate the record.

   a. Describe the assumptions required for your strategy to work correctly.

   b. Write pseudocode for the algorithm.

   c. Derive a recurrence equation for the worst-case running time.

   d. Solve the recurrence and express the complexity using asymptotic notation.

   e. Explain how your strategy would change if the dataset could not be divided into ordered groups.